

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
про виконання лабораторної роботи №2
“ Структури даних та функціональна декомпозиція ”
з курсу
“ Професійний Python”

Виконала:
Студентка групи ФЕП-33
Бойчук Карина
Перевірив:
Свелеба Сергій Андрійович

1. Мета роботи

Сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпозувати програму на невеликі функції з чітко визначеною відповідальністю.

У ході роботи необхідно опанувати:

- list;
- tuple;
- set;
- dict;
- модуль collections;
- list/set/dict comprehensions;
- генераторні вирази;
- оцінювання часової складності операцій;
- функціональну декомпозицію;
- позиційні та іменовані аргументи;
- *args;
- **kwargs;
- lambda;
- closures;
- decorators.

Хід роботи:

Варіант 2. Аналіз системи обліку книг

4. Постановка задачі

Необхідно розробити Python-проект для роботи з колекцією книг.

Кожна книга представлена словником, який містить такі поля:

- id — унікальний ідентифікатор книги;
- title — назва книги;
- author — автор;
- year — рік видання;
- pages — кількість сторінок.

Програма повинна забезпечувати:

- зберігання набору книг;
- пошук книги за ідентифікатором;
- пошук за назвою або автором;
- фільтрацію книг за роком;
- визначення найбільшої книги;
- обчислення середньої кількості сторінок;
- сортування книг за роком;

- групування книг за авторами;
- підрахунок кількості книг кожного автора;
- отримання книг конкретного автора;
- використання list, tuple, set, dict;
- використання collections;
- використання comprehensions;
- демонстрацію *args, **kwargs, lambda, closure та decorator;
- порівняння швидкості пошуку у list та dict.

5. Опис структури даних

Основною структурою даних програми є список list, елементами якого є словники dict.

Приклад одного елемента:

```
{  
    "id": 1,  
    "title": "The Hobbit",  
    "author": "J.R.R. Tolkien",  
    "year": 1937,  
    "pages": 310,  
}
```

Структура:

```
1000000 0.000638280 0.000000210
PS D:\3\python\Lab_02\lab_structures> tree /F
Folder PATH listing
Volume serial number is CA99-2AAC
D:
├── pyproject.toml
├── README.md
├── src
│   ├── data_processor
│   │   ├── analytics.py
│   │   ├── benchmark.py
│   │   ├── data.py
│   │   ├── decorators.py
│   │   ├── main.py
│   │   ├── processors.py
│   │   └── __init__.py
│   └── __pycache__
│       ├── analytics.cpython-314.pyc
│       ├── benchmark.cpython-314.pyc
│       ├── data.cpython-314.pyc
│       ├── decorators.cpython-314.pyc
│       ├── main.cpython-314.pyc
│       ├── processors.cpython-314.pyc
│       └── __init__.cpython-314.pyc
PS D:\3\python\Lab_02\lab_structures>
```

Також у програмі використовуються:

- list — для зберігання колекції книг та результатів фільтрації;
- tuple — для зберігання незмінної інформації про книгу та статистики;
- set — для отримання унікальних авторів та років;
- dict — для представлення книги та створення індексу книг за id.

6. Пояснення вибору list, tuple, set, dict

List

list використовується як основна структура для зберігання всіх книг.

```
books = [
    {...},
    {...},
]
```

Список зручний, оскільки книги потрібно перебирати, фільтрувати та сортувати.

Tuple

tuple використовується для зберігання набору характеристик книги:

```
first_book_info = (  
    books[0]["title"],  
    books[0]["year"],  
    books[0]["pages"],  
)
```

Кортеж підходить для даних, які після створення не потрібно змінювати.

Також функція `get_page_statistics()` повертає кортеж:

```
return (  
    min(pages),  
    max(pages),  
    sum(pages) / len(pages),  
)  
Set
```

set використовується для отримання унікальних авторів:

```
return {  
    book["author"]  
    for book in books  
}
```

Множина автоматично видаляє дублікати, тому кожен автор зберігається лише один раз.

Dict

dict використовується для представлення кожної книги.

Наприклад:

```
{  
    "id": 1,  
    "title": "The Hobbit",  
    "author": "J.R.R. Tolkien",  
    "year": 1937,  
    "pages": 310,  
}
```

Також створюється словник-індекс:

```
{
```

```
    book["id"]: book
    for book in books
}
```

Це дозволяє швидко отримувати книгу за її id.

7. Приклад використання collections

У програмі використовується модуль collections, зокрема Counter та defaultdict.

Counter

Counter використовується для підрахунку кількості книг кожного автора:

```
return Counter(
    book["author"]
    for book in books
)
```

Наприклад, для автора George Orwell результат містить кількість його книг.

defaultdict

defaultdict(list) використовується для групування книг за авторами:

```
result = defaultdict(list)
```

```
for book in books:
    result[book["author"]].append(book)
```

Перевагою defaultdict є те, що для нового автора автоматично створюється порожній список.

9. Основні функції

У програмі реалізовано такі основні функції.

```
get_unique_authors()
```

Повертає множину унікальних авторів:

```
def get_unique_authors(books: list[dict]) -> set[str]:
    return {
        book["author"]
        for book in books
    }
```

`create_book_index()`

Створює словник-індекс, де ключем є id книги:

```
def create_book_index(books: list[dict]) -> dict[int, dict]:  
    return {  
        book["id"]: book  
        for book in books  
    }  
filter_by_year()
```

Фільтрує книги за діапазоном років.

`search_book()`

Виконує пошук за назвою або автором.

`find_largest_book()`

Знаходить книгу з найбільшою кількістю сторінок.

`calculate_average_pages()`

Обчислює середню кількість сторінок.

`sort_by_year()`

Сортує книги за роком видання.

`group_books_by_author()`

Групує книги за авторами.

`count_books_by_author()`

Підраховує кількість книг кожного автора.

`get_books_by_author()`

Повертає книги вказаного автора.

10. Приклад comprehensions

У програмі використано три типи comprehensions.

List comprehension

Отримання списку назв книг:

```
titles = [  
    book["title"]  
    for book in books  
]  
Set comprehension
```

Отримання унікальних років:

```
years = {  
    book["year"]  
    for book in books  
}  
Dict comprehension
```

Створення словника, де ключем є назва книги, а значенням — кількість сторінок:

```
pages_by_title = {  
    book["title"]: book["pages"]  
    for book in books  
}
```

Comprehensions дозволяють записати операції створення колекцій у компактній формі.

11. Приклад *args

У програмі реалізована функція:

```
def calculate_average_values(  
    *values: float,  
) -> float:  
    if not values:  
        return 0.0  
  
    return sum(values) / len(values)
```

*args дозволяє передати функції довільну кількість позиційних аргументів.

Приклад:

```
calculate_average_values(  
    100,  
    200,  
    300,  
    400,  
)
```

Результат:

250.0

12. Приклад `**kwargs`

Для демонстрації `**kwargs` використовується функція:

```
def create_record(  
    **fields,  
) -> dict:  
    return dict(fields)
```

Виклик:

```
new_book = create_record(  
    id=13,  
    title="Clean Code",  
    author="Robert C. Martin",  
    year=2008,  
    pages=464,  
)
```

Результатом є словник:

```
{  
    "id": 13,  
    "title": "Clean Code",  
    "author": "Robert C. Martin",  
    "year": 2008,  
    "pages": 464  
}
```

`**kwargs` дозволяє передавати довільну кількість іменованих аргументів.

13. Приклад `lambda`

Lambda-функції використовуються під час пошуку максимальної кількості сторінок та сортування.

Приклад:

```
max(  
    books,  
    key=lambda book: book["pages"],  
)
```

Тут `lambda` повертає кількість сторінок конкретної книги.

Також lambda використовується при сортуванні:

```
sorted(  
    books,  
    key=lambda book: book["year"],  
    reverse=reverse,  
)
```

У цьому випадку книги сортуються за роком видання.

14. Пояснення closure

У програмі використовується замикання у функції `create_page_filter()`.

```
def create_page_filter(  
    minimum_pages: int,  
):  
    def predicate(book: dict) -> bool:  
        return book["pages"] >= minimum_pages  
  
    return predicate
```

Спочатку створюється фільтр:

```
is_large_book = create_page_filter(400)
```

Після цього він використовується:

```
large_books = filter_items(  
    books,  
    is_large_book,  
)
```

Функція `predicate` запам'ятовує значення `minimum_pages`, яке було передано під час створення фільтра.

Це і є `closure` (замикання) — внутрішня функція зберігає доступ до змінної із зовнішньої функції навіть після завершення її виконання.

15. Пояснення decorator

У програмі використовується декоратор `measure_time`.

```
def measure_time(func):  
    @wraps(func)  
    def wrapper(*args, **kwargs):  
        start = perf_counter()
```

```
result = func(*args, **kwargs)
```

```
elapsed = perf_counter() - start
```

```
print(  
    f"{func.__name__}: "  
    f"{elapsed:.8f} s"  
)
```

```
return result
```

```
return wrapper
```

Декоратор підключається до функції:

```
@measure_time  
def calculate_average_pages(...):
```

Його призначення — виміряти час виконання функції.

Алгоритм роботи:

1. Фіксується час початку.
2. Виконується оригінальна функція.
3. Фіксується час завершення.
4. Обчислюється різниця.
5. Час виводиться на екран.
6. Повертається результат функції.

У програмі також реалізовано декоратор `repeat`, який дозволяє виконати функцію задану кількість разів.

16. Результат роботи програми

Під час запуску програма виводить інформацію про книги та результати виконання операцій.

Основні результати:

- кількість книг — 12;
- найбільша книга — Emma, 474 сторінки;
- найменша книга — The Little Prince, 96 сторінок;
- середня кількість сторінок — приблизно 273.75;
- виконується пошук книги за id;
- виконується пошук за назвою або автором;
- виконується фільтрація за роком;
- книги групуються за авторами;
- підраховується кількість книг кожного автора;

- створюються списки, множини та словники за допомогою comprehensions.

Результати:

```
PS D:\3\python\Lab_02\lab_structures> py -m src.data_processor.main
```

All books

1	The Hobbit	J.R.R. Tolkien	1937	310
2	1984	George Orwell	1949	328
3	Animal Farm	George Orwell	1945	112
4	Pride and Prejudice	Jane Austen	1813	432
5	Emma	Jane Austen	1815	474
6	Harry Potter and the Philosopher's Stone	J.K. Rowling	1997	223
7	Harry Potter and the Chamber of Secrets	J.K. Rowling	1998	251
8	The Great Gatsby	F. Scott Fitzgerald	1925	180
9	To Kill a Mockingbird	Harper Lee	1960	336
10	The Catcher in the Rye	J.D. Salinger	1951	277
11	Fahrenheit 451	Ray Bradbury	1953	256
12	The Little Prince	Antoine de Saint-Exupery	1943	96

LIST:

```
<class 'list'>
```

Number of books: 12

TUPLE:

```
('The Hobbit', 1937, 310)
```

Unique authors:

Antoine de Saint-Exupery

F. Scott Fitzgerald

George Orwell

Harper Lee

J.D. Salinger

J.K. Rowling

J.R.R. Tolkien

Jane Austen

Ray Bradbury

Dictionary index:

```
{'id': 1, 'title': 'The Hobbit', 'author': 'J.R.R. Tolkien', 'year': 1937, 'pages': 310}
```

Search by ID:

Search by ID:
{'id': 5, 'title': 'Emma', 'author': 'Jane Austen', 'year': 1815, 'pages': 474}

Search result: Harry

6	Harry Potter and the Philosopher's Stone	J.K. Rowling	1997	223
7	Harry Potter and the Chamber of Secrets	J.K. Rowling	1998	251

Books from 1900 to 1950

1	The Hobbit	J.R.R. Tolkien	1937	310
2	1984	George Orwell	1949	328
3	Animal Farm	George Orwell	1945	112
8	The Great Gatsby	F. Scott Fitzgerald	1925	180
12	The Little Prince	Antoine de Saint-Exupery	1943	96

Largest book:
Emma - 474 pages
calculate_average_pages: 0.00001750 s

Average number of pages: 272.92

Books sorted by year

4	Pride and Prejudice	Jane Austen	1813	432
5	Emma	Jane Austen	1815	474
8	The Great Gatsby	F. Scott Fitzgerald	1925	180
1	The Hobbit	J.R.R. Tolkien	1937	310
12	The Little Prince	Antoine de Saint-Exupery	1943	96
3	Animal Farm	George Orwell	1945	112
2	1984	George Orwell	1949	328
10	The Catcher in the Rye	J.D. Salinger	1951	277
11	Fahrenheit 451	Ray Bradbury	1953	256
9	To Kill a Mockingbird	Harper Lee	1960	336
6	Harry Potter and the Philosopher's Stone	J.K. Rowling	1997	223
7	Harry Potter and the Chamber of Secrets	J.K. Rowling	1998	251

Books grouped by author:

J.R.R. Tolkien: 1 book(s)

Books grouped by author:

J.R.R. Tolkien: 1 book(s)
George Orwell: 2 book(s)
Jane Austen: 2 book(s)
J.K. Rowling: 2 book(s)
F. Scott Fitzgerald: 1 book(s)
Harper Lee: 1 book(s)
J.D. Salinger: 1 book(s)
Ray Bradbury: 1 book(s)
Antoine de Saint-Exupery: 1 book(s)

Counter books by author:
Counter({'George Orwell': 2, 'Jane Austen': 2, 'J.K. Rowling': 2, 'J.R.R. Tolkien': 1, 'F. Scott Fitzgerald': 1, 'Harper Lee': 1, 'J.D. Salinger': 1, 'Ray Bradbury': 1, 'Antoine de Saint-Exupery': 1})

Books by George Orwell

2	1984	George Orwell	1949	328
3	Animal Farm	George Orwell	1945	112

List comprehension:
['The Hobbit', '1984', 'Animal Farm', 'Pride and Prejudice', 'Emma', 'Harry Potter and the Philosopher's Stone', 'Harry Potter and the Chamber of Secrets', 'The Great Gatsby', 'To Kill a Mockingbird', 'The Catcher in the Rye', 'Fahrenheit 451', 'The Little Prince']

Set comprehension:
[1813, 1815, 1925, 1937, 1943, 1945, 1949, 1951, 1953, 1960, 1997, 1998]

Dict comprehension:
The Hobbit: 310 pages
1984: 328 pages
Animal Farm: 112 pages
Pride and Prejudice: 432 pages
Emma: 474 pages
Harry Potter and the Philosopher's Stone: 223 pages
Harry Potter and the Chamber of Secrets: 251 pages
The Great Gatsby: 180 pages
To Kill a Mockingbird: 336 pages
The Catcher in the Rye: 277 pages

```

Harry Potter and the Chamber of Secrets: 251 pages
The Great Gatsby: 180 pages
To Kill a Mockingbird: 336 pages
The Catcher in the Rye: 277 pages
Fahrenheit 451: 256 pages
The Little Prince: 96 pages

Average via *args: 250.0

Created via **kwargs:
{'id': 13, 'title': 'Clean Code', 'author': 'Robert C. Martin', 'year': 2008, 'pages': 464}

Books with 400 or more pages
-----
4 Pride and Prejudice          Jane Austen          1813  432
5 Emma                        Jane Austen          1815  474

Page statistics:
Minimum: 96
Maximum: 474
Average: 272.92

Benchmark
-----
Records      List search      Dict search
1000         0.00006040       0.00000170
10000        0.00058340       0.00000270
100000       0.00593590       0.00000230
PS D:\3\python\Lab_02\lab_structures> |

```

17. Таблиця складності операцій

Операція	Структура	Середня складність
Доступ за індексом	list	$O(1)$
Пошук елемента	list	$O(n)$
Додавання в кінець	list	$O(1)$ амортизована
Перевірка належності	set	$O(1)$
Додавання елемента	set	$O(1)$
Пошук за ключем	dict	$O(1)$
Додавання елемента	dict	$O(1)$
Пошук у dict	dict	$O(1)$
Пошук максимуму	list	$O(n)$
Фільтрація	list	$O(n)$
Сортування	list	$O(n \log n)$
Створення Counter	Counter	$O(n)$
Групування	defaultdict	$O(n)$

Тут n — кількість елементів у колекції.

Важливо, що пошук у списку потребує послідовного перегляду елементів, тоді як словник використовує хеш-таблицю, тому пошук за ключем у середньому виконується за $O(1)$.

18. Результати benchmark

У програмі реалізовано benchmark для порівняння пошуку книги у списку та словнику.

Для тестування використовуються набори:

```
sizes = [  
    1000,  
    10000,  
    100000,  
]
```

Для списку використовується лінійний пошук:

```
def find_linear(books, book_id):  
    for book in books:  
        if book["id"] == book_id:  
            return book  
  
    return None
```

Для словника використовується:

```
index.get(book_id)
```

Результат benchmark виводиться у вигляді таблиці:

Benchmark

Records	List search	Dict search
1000	...	...
10000	...	...
100000	...	...

19. Аналіз результатів

У ході виконання лабораторної роботи було продемонстровано використання основних вбудованих структур даних Python.

Список використовується для зберігання колекції книг, словник — для представлення окремої книги та створення індексу, множина — для отримання унікальних значень, а кортеж — для зберігання незмінних наборів значень.

Використання Counter спрощує підрахунок книг за авторами, а defaultdict дозволяє зручно групувати книги.

Функціональні можливості Python продемонстровано за допомогою comprehensions, lambda-функцій, *args, **kwargs, closure та decorators.

Benchmark дозволяє практично порівняти два способи пошуку. Лінійний пошук у списку має складність $O(n)$, а пошук за ключем у словнику в середньому має складність $O(1)$.

20. Висновок

У результаті виконання лабораторної роботи було отримано практичні навички роботи з основними структурами даних Python: list, tuple, set та dict.

Було реалізовано програму для обробки колекції книг, яка підтримує пошук, фільтрацію, сортування, групування та статистичний аналіз даних.

Також було використано можливості модуля collections, зокрема Counter і defaultdict. На практиці продемонстровано використання list, set та dict comprehensions, lambda-функцій, *args, **kwargs, closure та decorators.

Окремо було проведено benchmark пошуку у списку та словнику. Теоретичний аналіз показує, що лінійний пошук у списку має складність $O(n)$, тоді як пошук за ключем у словнику в середньому має складність $O(1)$. Це демонструє важливість правильного вибору структури даних залежно від задачі.

Таким чином, поставленої мети роботи було досягнуто.